Cpts 327 Lab 1 Quiz

Instructions

- **Format Requirement:** Please round (nor floor or ceil) all numerical answers to **two decimal places** (Please use fractions in intermediate steps to avoid precision loss and only round at the very end), e.g., round the result 4.555 to 4.56, 4.444 to 4.44. If your calculated answer has fewer than two decimal places (e.g., 0.1), you must pad it with a trailing zero (e.g., enter **0.10**) to ensure the grading system accepts it.

Question 1: (3 Points) A buffer overflow occurs when a program writes data beyond the allocated memory boundary.

- **A. True.**
- **B. False.**

Question 2: (3 Points) The function **gets()** checks input length to prevent overflow.

- **A. True.**
- **B. False.**

Question 3: (3 Points) Which function is most likely to cause a buffer overflow?

- **A.** fgets
- **B.** gets
- **C.** strcpy
- **D.** strlen

Question 4: (3 Points) Using **scanf("%s", buf)** may cause buffer overflow if input is too long.

- **A. True.**
- **B. False.**

Question 5: (3 Points) Which statement about **strncpy** is correct?

- **A.** Always safe
- **B.** Always null-terminates the string
- **C.** May not append a null terminator
- **D.** Does not limit copy length

Question 6: (3 Points) Buffer overflow can only occur on the stack.

- **A. True.**
- **B. False.**

Question 7: (3 Points) Which of the following is NOT a typical consequence of buffer overflow?

- **A.** Program crash
- **B.** Data corruption
- **C.** Arbitrary code execution
- **D.** Automatic memory repair

Question 8: (6 Points) Which statement is correct for the code below?

```
char buf[10];
fgets(buf, 10, stdin);
```

- **A.** It will always overflow
- **B.** It reads no input
- **C.** It reads at most 9 characters (or until a newline) and always appends a null terminator
- **D.** It behaves the same as gets

Question 9: (6 Points) An attacker can overwrite the return address via buffer overflow to control program execution.

- **A. True.**
- **B. False.**

Question 10: (7 Points) Consider the following code running on a system with NX and ASLR enabled, but no stack canary:

```
void vulnerable() {
    char buf[64];
    gets(buf);
}
```

If an attacker successfully exploits this vulnerability, what is the most likely attack method?

- **A.** Inject shellcode into the stack and execute it directly
- **B.** Overwrite return address and redirect execution to existing code (e.g., libc)
- **C.** The system will automatically prevent exploitation
- **D.** It can only cause a crash, not an exploit